

VDDPI: Verifiable Decentralized Data Processing Infrastructure for Data Usage Control and Confidential Data Processing

Shota Tokuda¹, Shohei Kakei¹, Yoshiaki Shiraishi¹, *Member, IEEE*, and Shoichi Saito¹

Abstract—The utilization of personal data can create new insights. On the other hand, data providers have concerns about unauthorized acquisition and misuse of provided data. We propose a decentralized data processing infrastructure, named VDDPI, that enables data providers to verify data processing implemented by data consumers. VDDPI manages information on data processing apps using distributed ledger technology, and data providers can obtain this information and check whether the provided data will be disclosed or not before providing data. Furthermore, by running the app using trusted execution environments (TEEs), the integrity of the app and the confidentiality of the data provided are ensured. Expanding beyond data processing verification, VDDPI provides decentralized data usage control, allowing continuous control over the number of uses, location, date and time of use. We evaluated whether VDDPI meets security requirements derived from STRIDE and performed formal verification using ProVerif. In addition, we implemented a prototype using Intel SGX and Hyperledger Fabric and compared its performance with related work. The results of the performance evaluation confirmed that the data processing time was approximately equivalent to related work.

Index Terms—Data utilization, data usage control, distributed ledger technology, formal verification, trusted execution environment (TEE).

I. INTRODUCTION

THE use of personal data is gaining attention in a variety of fields. In the context of the coronavirus disease 2019 (COVID-19) pandemic, personal data were employed in the development of public health measures [1]. In the context of health care, services can also be made more efficient by utilizing personal health data [2], [3].

In a centralized model, where personal data is transferred to a central organization such as a government or municipality for data processing, the individual data provider is concerned about privacy violations [4], [5]. In response to the COVID-19 pandemic, the centralized architecture was considered to track contacts and calculate infection status, but this architecture was

ultimately not adopted in many countries due to such concerns [6], [7].

Data sovereignty, a concept related to the ability of individuals to control the use of their personal data for the purpose of protecting their privacy, is receiving attention [8]. One method of achieving data sovereignty is to utilize personal data stores (PDSs) [9]. PDSs manage personal data by individuals themselves on their own devices and servers, thereby mitigating the risk of privacy violations. However, when data consumers intend to process data, they should perform the processing on their own systems using their own algorithms. This data processing may result in the unrestricted use of the provided data. In order for data providers to be able to provide personal data without privacy concerns in such a scenario, it is essential to have a mechanism that allows them to establish trust with the data processor that runs on the untrusted data consumers.

Audit-oriented frameworks have been studied as a means to increase accountability in data usage by recording how data and applications were used in an immutable and transparent manner. Nascimento et al. [10] proposed DNAT, which integrates Ethereum and Intel SGX to provide publicly auditable usage records. In DNAT, data and application providers can register assets on a blockchain, and trusted nodes mediate access requests to ensure that usage agreements are properly recorded.

Beyond audit-oriented frameworks, usage control has been studied as an approach that provides stronger guarantees by technically enforcing conditions before and during data usage. Park et al. [11] introduced data usage control to continuously monitor digital resources in dynamic distributed environments. It extends general access control by enabling more detailed data control after data are provided. Recent research has been conducted on methods to enforce compliance with data usage control in distributed environments [12], [13], [14]. This research uses a trusted execution environment (TEE) to achieve compliance with usage conditions through data usage control. Some work checks data processing code to prevent data misuse [15], [16], [17], [18]. However, this verification relies on assumptions or a single point of failure, risking misuse if the central authority is compromised. Another approach involves releasing the code for verification by data providers [19], [20], [21], but companies are unwilling to disclose the code due to its intellectual property rights [22]. Thus, it has been considered that it is difficult for data providers to properly verify programs implemented by data consumers running in a remote TEE enclave [23].

Received 15 March 2025; revised 11 January 2026; accepted 20 January 2026. Date of publication 4 February 2026; date of current version 12 May 2026. This work was supported by JSPS KAKENHI under Grant JP22K17881, Grant JP25K21199, and Grant JP23K03847. (*Corresponding author: Shohei Kakei.*)

Shota Tokuda, Shohei Kakei, and Shoichi Saito are with the Graduate School of Engineering, Nagoya Institute of Technology, Nagoya, Aichi 466-8555, Japan (e-mail: s.tokuda.570@nitech.jp; kakei.shohei@nitech.ac.jp; shoichi@nitech.ac.jp).

Yoshiaki Shiraishi is with the Graduate School of Engineering, Kobe University, Kobe, Hyogo 657-8501, Japan (e-mail: zenmei@port.kobe-u.ac.jp).

Digital Object Identifier 10.1109/TDSC.2026.3658309

The processing infrastructure that handles sensitive data such as personal data behaves as a trusted system, so it should be secure and dependable, and should guarantee the following properties: confidentiality, integrity, reliability, and maintainability and more [24]. By realizing such a secure data infrastructure, concerns about the confidentiality of provided data can be eliminated, thereby promoting the utilization of personal data.

This paper introduces a verifiable decentralized data processing infrastructure, VDDPI, using TEE and distributed ledger technology (DLT). This infrastructure uses TEE to ensure the security of provided data and data processing processes, and guarantees dependability using blockchain technology. VDDPI addresses security requirements in a decentralized data processing model assumed in this study to counter threats identified in STRIDE. Data consumers implement data processing codes based on a template specified by VDDPI. This template contains codes for communication with data providers and data usage control, and the security baseline for data processing is assured by verifying that the implemented source code is compliant with the template. In addition, data flow analysis is performed to prevent unintended data leakage. VDDPI uses smart contracts to ensure the integrity of the verification and analysis process. VDDPI enables data providers to verify enclave programs running in remote TEE enclaves, combining flexible data processing with data protection.

We previously proposed a data usage control method in a decentralized data processing model [25]. However, various threats exist in the actual operation of the system. Therefore, in this paper, we propose a data processing infrastructure based on our method that mitigates these threats. In addition, we conduct a more rigorous security and performance evaluation of VDDPI.

The contributions of this paper are as follows:

- We identify potential threats in a decentralized data processing model using STRIDE where data is transferred between data providers and data consumers and define security requirements in this model.
- We propose a verifiable decentralized data processing infrastructure, named VDDPI. VDDPI is designed based on the security requirements derived from STRIDE threat modeling. VDDPI enables data providers to verify the data processing performed in a TEE enclave in the cloud. As a result, VDDPI prevents the unauthorized use and acquisition of provided data while enabling execution of data processing implemented by data consumers.
- We evaluate that VDDPI meets each requirement and formally verify the protocol using ProVerif, a formal verification tool, to demonstrate the safety of VDDPI.
- We implement a prototype of VDDPI using Intel SGX and Hyperledger Fabric to demonstrate its feasibility, and publish the source code on Github.¹ This prototype is used to compare execution time with that of a related study.

The remainder of this paper is structured as follows. Section II provides background and assumed data processing model. Section III describes threat modeling and security requirements. Section IV introduces a data usage control method proposed

in our previous study. Section V describes VDDPI, a data processing infrastructure. Section VI and Section VII describe the security evaluation, implementation, and performance evaluation of VDDPI. Section VIII compares VDDPI with related work. Section IX discusses VDDPI, and Section X concludes.

II. BACKGROUND

A data utilization model involves data providers depositing data with a central authority, such as a broker, allowing data consumers to gain insights. This centralized data processing model is the dominant model for online services today, where a single organization collects and stores information on multiple individuals [26]. This model forces data providers to trust that the central authority will not misuse their data. If this central authority is compromised, the data could be misused. As a data infrastructure develops, increasing data types and volumes heighten leakage risks, creating a dilemma [27]. This dilemma requires a choice: accept the risk to scale services or limit data to reduce risk. A decentralized approach mitigates this risk by enabling direct data exchange between providers and consumers.

In a model where data providers and consumers communicate directly, it is important to protect the provided data in-use, at-rest, and in-transit. TEE can help achieve this protection, but unauthorized processing by untrusted data consumers may compromise provided data. Several studies have proposed systems to verify data processing in TEE enclaves [15], [16], [17], [19], [20], [21]. However, as the size of code increases, the verification by data providers or third parties becomes infeasible [23]. Furthermore, data consumers may require the ability to implement processing beyond predefined code [28]. It is therefore necessary to incorporate a mechanism to mechanically verify the data processing implemented by data consumers. In addition, continuous control of provided data is important for data protection in a decentralized data processing model. In a decentralized data processing model where data consumers cannot be trusted, technical assurance of continuous data control is important. However, research on dependable guarantees for data sovereignty remains limited [29].

A. Motivating Use Case

We assume that individuals manage their data and provide data as needed and develops a generic infrastructure that can be applied to various cases. However, we focus on a specific use case in this paper. Fig. 1 shows a hypothetical use case in which data collected from personal smartphones is utilized to support evacuation guidance in the event of a disaster within a facility. In normal times, a facility as a data consumer can request and store personal data from PDSs via data processor in the cloud environment. While the facility calculates evacuation routes based on real-time damage assessments, uniform instructions may not be suitable for all visitors - such as wheelchair users who cannot use staircases. Furthermore, if health conditions, age, or other personal attributes are recorded in advance, first responders can leverage this information to assist in lifesaving activities. In addition, the use of location information obtained from smartphones enables rapid lifesaving activities. From a

¹<https://github.com/VDDPI/VDDPI>

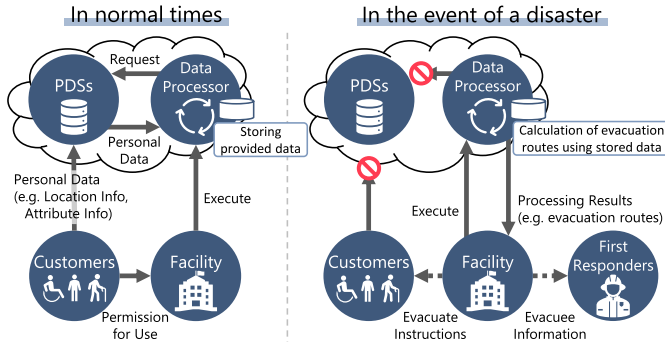


Fig. 1. Use case for disaster evacuation support.

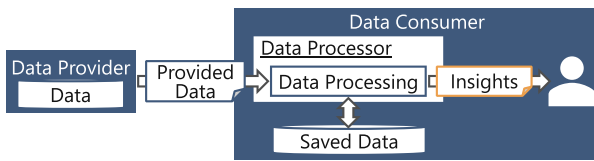


Fig. 2. Data processing model assumed in this paper.

disaster risk reduction perspective, collecting such attribute data in advance is beneficial.

In general, the collection of personal information during normal times allows data consumers to utilize the data at the desired time—such as during a disaster—without the need to interact with each individual data provider. This pre-collection can reduce the communication load caused by each data processor communicating simultaneously with the PDSs. However, the collection of data by data consumers can result in a loss of control by data providers over their data, thereby giving rise to privacy concerns. For example, personal location information may be leaked or used by unauthorized applications. Therefore, it is important to prevent unauthorized acquisition and misuse of the provided data in order to avoid privacy concerns of the data provider in such use cases. The objective of the infrastructure we propose is to prevent the unintended disclosure and unrestricted use of provided data in use cases where such personal data is utilized.

B. Assumed Data Processing Model

We define a decentralized data processing model as one in which data consumers execute data processing on their own systems, and we adopt this model for the data processing assumed in this study. The data processing model assumed in this study is shown in Fig. 2. The data processor in this model may be running in the data consumer's local environment, or it may be running in the cloud environment used by the data consumer. In this model, it is assumed that the data consumer implements the data processing and the data consumer can temporarily store the provided data for flexibility in data use. This temporary storage enables data processing even when the

data consumer cannot establish communication with the data provider.

In this model, data providers need to send data to untrusted data consumers directly. It is therefore essential to implement a continuous control mechanism to prevent the misuse of the provided data. The use of TEEs is gaining attention as a way to technically ensure continuous control [29]. Specifically, data providers provide data consumers with data usage policies that include usage conditions. This policy is stored with the provided data in order to enforce it when the data is reused.

C. Trusted Execution Environment

It is important to ensure data confidentiality during data use when processing sensitive data in a cloud environment or an external environment. One way to ensure the confidentiality of data during processing is a trusted execution environment (TEE). It ensures the authenticity of the enclave code, the integrity of the runtime states, and the confidentiality of its code and data [30]. In this study, we opted for Intel Software Guard Extensions (SGX) [31] as a TEE. SGX creates an encrypted protected area in memory called *enclaves*. Applications running within an enclave can be uniquely identified by *MRENCLAVE*, a hash value that depends on the enclave program and configuration files. SGX also has a sealing mechanism that stores confidential data in non-volatile memory encrypted with an enclave program-specific key [32]. This mechanism enables data reuse with confidentiality.

Remote attestation is used to attest the legitimacy and integrity of an enclave program and platform by presenting an attestation report, including *MRENCLAVE*, to remote entities. By checking *MRENCLAVE*, remote entities can verify that the program in an enclave with which they communicate is their intended program. RA-TLS [33] is a method for embedding remote attestation in a TLS negotiation.

D. Distributed Ledger Technology

It is important to guarantee data integrity when sharing data. Distributed ledger technology (DLT) is a method of ensuring integrity by storing data across multiple distributed nodes. DLT allows multiple participants to maintain synchronized ledgers on a network without a central administrator. This technology guarantees transaction transparency and data integrity. In this research, we opted for Hyperledger Fabric [34], permissioned DLT. Permissionless DLT is open to anyone, and each participant is anonymous, whereas permissioned DLT consists of participants approved by an administrator.

III. THREAT MODELING AND REQUIREMENTS

In this section, we use STRIDE to identify threats to the protection of provided data from adversaries assumed in the data processing model. First, the adversary is described, and then the assumed data processing model is converted to a data flow diagram (DFD). Next, STRIDE is used to identify potential threats and consider possible attack scenarios for each threat.

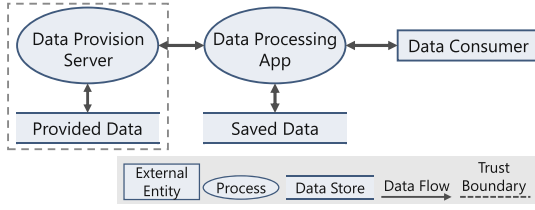


Fig. 3. Data flow diagram of assumed model.

Finally, we define requirements for protecting provided data from each attack.

A. Assumptions and Limitations

In this study, we assume that adversaries do not perform side-channel attacks against vulnerabilities in TEE. We also assume that the cryptographic primitives used in TLS and X.509 certificates are not compromised and are sufficiently secure. Additionally, we assume that private keys are not compromised. This study does not cover the confidentiality of processing results after they are obtained by the data consumer.

In addition, our design assumes that the data processing code is written in a restricted form so that the registry's verification process can inspect the code and apply static taint analysis. In particular, the code is expected to follow the template and programming constraints imposed by VDDPI, and leakage prevention relies on allowlisted functions such as sanitizer functions and other permitted operations. We therefore assume that allowlisted functions are correctly implemented for their intended purpose and that the allowlisting procedure, including manual review, is conducted appropriately. Due to these constraints, VDDPI supports existing data processing apps only to the extent that they can be adapted to our template and programming restrictions; consequently, porting legacy applications may require refactoring or partial reimplementations to meet these restrictions. Moreover, because the verification process needs sufficient visibility into the processing program to check policy compliance and potential leakage, the confidentiality of the processing code is only partially preserved; therefore, scenarios that require keeping the full processing code strictly secret from external parties are not supported in the current design.

We also rely on components outside the enclave trust boundary to operate as intended. Specifically, we assume that the permissioned distributed ledger platform, the certificate authority for authentication, and the software toolchain used to build and analyze the processing code are not compromised in a way that breaks the integrity of verification or the correctness of access control. While some nodes may be faulty or compromised, we assume that a data processor can mitigate this risk by consulting multiple registry nodes and by validating ledger records through the platform's consensus mechanism. We do not address attacks that fundamentally undermine these external trust assumptions (e.g., compromises that subvert consensus or the integrity of the toolchain).

Finally, our design provides only limited protection against state rollback and certain classes of memory attacks. Although

sealing enables reuse of confidential state, it does not inherently prevent an adversary from rolling back sealed data to a previous version; thus, strong rollback protection requires additional platform support (e.g., trusted monotonic counters) or is treated as a limitation. Furthermore, beyond the side-channel attacks excluded above, we do not guarantee confidentiality if vulnerabilities in the TEE implementation or privileged software enable direct disclosure of enclave memory; such attacks are considered out of scope and remain a limitation of the protection offered by this study.

B. Assumed Adversaries

As discussed in Section II, decentralized data processing models may lead to reluctance to share data because of losing control over provided data. This is because data providers are concerned that untrustworthy data consumers may fraudulently obtain or misuse the data provided. Therefore, this study designs an infrastructure that addresses these concerns. Specifically, we assume the following five types of attacks.

- UA-1: Unauthorized acquisition of provided data
- UA-2: Unauthorized acquisition of processing results
- UU-1: Failure to comply with data usage policies
- UU-2: Data use by unauthorized data consumers
- UU-3: Data use by unauthorized data processing app

UA-1 and UA-2 represent attacks related to the unauthorized acquisition of data. These attacks result in the disclosure of data to which the data provider does not consent. In UU-1, UU-2, and UU-3, data consumers perform unauthorized use of provided data without consideration of the terms agreed upon by data providers at the time of its provision.

C. Stride

We use STRIDE [35] for threat analysis of the data processing model assumed in this study. STRIDE is a threat modeling methodology developed by Microsoft that can analyze six types of threats (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service (DoS), and Elevation of Privilege (EoP)). In this paper, we employ STRIDE-per-Element [36], which applies STRIDE to each element of a DFD, to perform threat analysis.

D. Threat Modeling

First, to identify potential threats, we convert the assumed data processing model to a DFD. The converted DFD is shown in Fig. 3. The DFD consists of four types of elements (External Entity, Process, Data Store, and Data Flow). A data provider has a data provision server and a database that stores provided data and communicates between these entities. In this study, to analyze threats from data providers' perspective, the data provider and the data store are surrounded by a trusted boundary. On the data consumer side, there is a data processing app and a data store for provided data, and a data consumer obtains processing results from the app. In addition, data are provided in the communication between the app and the server.

TABLE I
POTENTIAL THREATS OF DFD ELEMENTS IDENTIFIED BY STRIDE [35]
(† INDICATES AN EXCEPTIONAL CASE)

DFD Element Type	S	T	R	I	D	E
Data Store		✓	†	✓	✓	
Data Flow		✓		✓		
Process	✓	✓	✓	✓	✓	✓
External Entity	✓		✓			

† Exceptional case where a data store may face repudiation threats when used for logging or auditing, since log tampering may allow denial of past actions.

TABLE II
POTENTIAL THREATS OF ASSUMED DATA PROCESSING MODEL

DFD Element	Threat Target	S	T	R	I	D	E
Data Store	Saved Data			T-3	-	T-10	T-15
	Provided Data			-	-	-	-
Data Flow	Server-App			T-4		T-11	T-16
	App-Saved Data			T-5		T-12	T-17
	App-Data Consumer			T-6		T-13	T-18
	Provided Data-Server			-	-	-	-
Process	Data Processing App	T-1	T-7	T-8	T-14	T-19	T-20
	Data Provision Server	-	-	-	-	-	-
Entity	Data Consumer	T-2		T-9			

Next, we identify potential threats by referring to the possible threat components specified by STRIDE, as shown in Table I. The dagger mark for data store in this table means that it is associated with a threat related to repudiation if the data store is used for logging or auditing. Since the data store assumed in this study does not manage logging information, we ignore this dagger.

The results of identifying potential threats are shown in Table II. The rows with a gray background are elements within the trust boundary, so we assume that no threat exists. A potential threat identified in Table I is marked with a hyphen if it is within the trust boundary. As for data store repudiation, no threat is assumed to exist because the information to be stored is not log information.

We then consider attack scenarios associated with the 20 potential threats. The results are shown in Table III. Each scenario was based on the five types of attacks described in Section III-B and was examined with reference to the threat trees [35]. Spoofing attacks assume the use of provided data by unauthorized apps or data consumers. App spoofing may result in the use of provided data by unauthorized data processing apps, leading to improper data acquisition (T-1). In addition, spoofing of data consumers may result in the fraudulent acquisition of data processing results (T-2). Tampering with the communication channel or data store could cause policies to be unintended by data providers, resulting in loss of control over provided data (T-3–5). Tampering with the communication channel between the app and the data consumer may result in output of processing results to unintended data consumers (T-6). Furthermore, tampering with apps may result in ignoring policies or disclosing provided data that data providers do not consent (T-7, 9). Repudiation causes data consumers to use provided data under conditions that differ from those presented in advance (T-8, 9). This attack involves the use of data in an unauthorized app, which can lead to a disregard for the presented policy and the unauthorized acquisition of

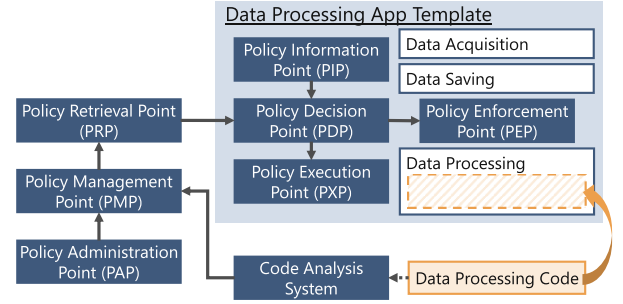


Fig. 4. Overview of proposed data usage control method [25].

data. Information disclosure in communication channels and data stores can lead to unauthorized acquisition of provided data (T-10–12). Information disclosure in the communication channel between data processing apps and data consumers can also lead to the leakage of processing results (T-13). In addition, information disclosure in apps can also occur from memory during app execution, which can lead to unauthorized acquisition of provided data (T-14). As for DoS, there are no attack scenarios in this study because they do not lead to the assumed attacks (T-15–19). EoP causes access to the data store by unauthorized apps, which leads to data usage by unintended apps (T-20).

E. Security Requirements

We define security requirements to prevent each of the attack scenarios described in Section III-D. The security requirements are shown in Table IV. As a result of the threat analysis, 12 security requirements were defined to prevent the five types of attacks in the assumed decentralized data processing model. Authentication of apps and data consumers is necessary to prevent spoofing (SR-1–3). Protection of data stores is also important to prevent policy tampering and ensure confidentiality of provided data (SR-4, 5). In addition, the protection of communication channels is required to prevent unintended disclosure of information (SR-6–8). Apps need to protect memory to prevent dynamic tampering during execution and the acquisition of provided data (SR-9), and to prevent tampering before execution (SR-10). To ensure non-repudiation, consistency between the data usage policy and the actual use of the data must be guaranteed (SR-11). In this paper, we design an infrastructure that technically satisfies this requirement. It is also important to verify data processing by the data provider, as apps may disclose provided data that data providers do not intend (SR-12).

IV. CODE VERIFIABLE DATA USAGE CONTROL METHOD

This section explains the data usage control method proposed in our previous study [25].

A. Overview

Fig. 4 provides an overview of the proposed data usage control method. This section describes the method using terms from XACML [37] and IDS [38]. In this method, data consumers must implement data processing apps following a specified

TABLE III
ATTACK SCENARIOS FOR EACH IDENTIFIED THREAT

Threat Type	ID	DFD Element	Overview of Attack Scenario	UA-1	UA-2	UU-1	UU-2	UU-3
Spoofing	T-1	Process (App)	Use of provided data in unauthorized apps	✓	✓	✓		✓
	T-2	External Entity (Consumer)	Use of provided data by unauthorized data consumers		✓		✓	
Tampering	T-3	Data Store (Saved Data)	Tampering with stored data usage policies			✓		
	T-4	Data Flow (Server–App)	Tampering with policies during communication, and providing data to unintended consumer or apps	✓	✓	✓	✓	✓
	T-5	Data Flow (App–Saved Data)	Tampering with policies during communication, and providing data to unintended consumer or apps	✓	✓	✓	✓	✓
	T-6	Data Flow (App–Consumer)	Results acquisition by unintended data consumer		✓			
	T-7	Process (App)	Tampering with apps such that policies are ignored or provided data are disclosed	✓	✓	✓		
Repudiation	T-8	Process (App)	Use of the provided data under different conditions from those at the time of the contract	✓	✓	✓	✓	✓
	T-9	External Entity (Consumer)						
Information Disclosure	T-10	Data Store (Saved Data)	Acquisition of provided data from saved data	✓				
	T-11	Data Flow (Server–App)	Acquisition of provided data during communication	✓				
	T-12	Data Flow (App–Saved Data)	Acquisition of provided data during communication	✓				
	T-13	Data Flow (App–Consumer)	Acquisition of processing results during communication or provided data as a processing result	✓	✓			
	T-14	Process (App)	Acquisition of data from memory during app running	✓				
DoS	T-15–19		DoS is not related to unauthorized acquisition or use.					
EoP	T-20	Process (App)	Access to stored data by unauthorized apps	✓	✓	✓		✓

TABLE IV
SECURITY REQUIREMENTS DERIVED FROM ATTACK SCENARIOS

Req.	Overview of Requirement	S	T	R	I	D	E	ID	UA-1	UA-2	UU-1	UU-2	UU-3
SR-1	Authentication of data processing apps by data providers	✓	✓					T-1, 3	✓	✓	✓		✓
SR-2	Authentication of data consumers by data providers	✓						T-2		✓		✓	
SR-3	Authentication of data consumers in data processing apps	✓	✓					T-2, 6		✓		✓	
SR-4	Protection of data (provided data and policies) in saved data		✓	✓				T-3, 10	✓		✓		
SR-5	Restriction of access to saved data		✓	✓	✓	✓		T-3, 10, 20	✓	✓	✓		✓
SR-6	Protection of the channel between the server and apps		✓	✓				T-4, 11	✓	✓	✓	✓	✓
SR-7	Protection of the channel between the app and saved data		✓	✓				T-5, 12	✓	✓	✓	✓	✓
SR-8	Protection of the channel between the app and the consumer		✓	✓				T-6, 13	✓	✓			
SR-9	Memory protection during execution of apps		✓	✓				T-3, 7, 14	✓	✓	✓		
SR-10	Prevention of tampering with data processing apps		✓	✓				T-3, 7, 14	✓	✓	✓		
SR-11	Ensuring consistency between actual use and data usage policy			✓				T-8, 9	✓	✓	✓	✓	✓
SR-12	Verification of data processing by data providers			✓				T-13	✓				

template, which defines data acquisition, temporary data storing, and a customizable data processing section. It also contains a policy decision point (PDP) that makes a decision on data usage based on a data usage policy provided by the policy retrieval point (PRP). The template includes a policy information point (PIP), a policy enforcement point (PEP), and a policy execution point (PXP) to ensure policy compliance in the app. The PRP acquires the instantiated data usage policy from the policy management point (PMP), which manages the policies created by the policy administration point (PAP). The PMP uses data disclosure information in the processing code to calculate usage conditions and confirm whether each input data is disclosed. This method enables continuous control of provided data by performing policy checks within data processing apps.

B. Processing Code Analysis

The proposed method employs data flow analysis to follow the flow of provided data in data processing. However, it could be difficult to analyze accurately. Therefore, as described in Section III-A, the method allows data consumers to implement only the data processing code, restricting the use of system calls and unauthorized third-party libraries that would complicate the analysis. On the other hand, calls to functions included in the allowlist are permitted. Then, data flow analysis is performed

by static taint analysis while maintaining code brevity through template compliance. It can be seen from the analysis that the provided data will not be disclosed unless any part of the provided data is returned from the data processing code.

The proposed method uses static taint analysis for data flow analysis of the data processing code within the code analysis system shown in Fig. 4. Static taint analysis is a type of data flow analysis that traces data transitions within a program to verify if data leaks have occurred. This analysis labels defined information sources, called *taint source*, and propagates the labels among the variables. Then, when a labeled variable reaches a defined point, called *taint sink*, the data corresponding to that label is found to be disclosed. In the taint analysis of the proposed method, the arguments of the data processing code are set as taint sources and the return value is set as a taint sink. By performing taint analysis in such a setting, it is possible to check whether provided data will be disclosed.

Provided data used as input in data processing affect the processing results in some way. Even if some of the provided data are modified, it is difficult to determine the extent to which such modifications result in anonymization. This data flow analysis prohibits the use of unauthorized third-party functions and system calls, while allowing the use of authorized functions. These authorized functions are intended to perform anonymization, such as calculating averages, and act as *sanitizer*

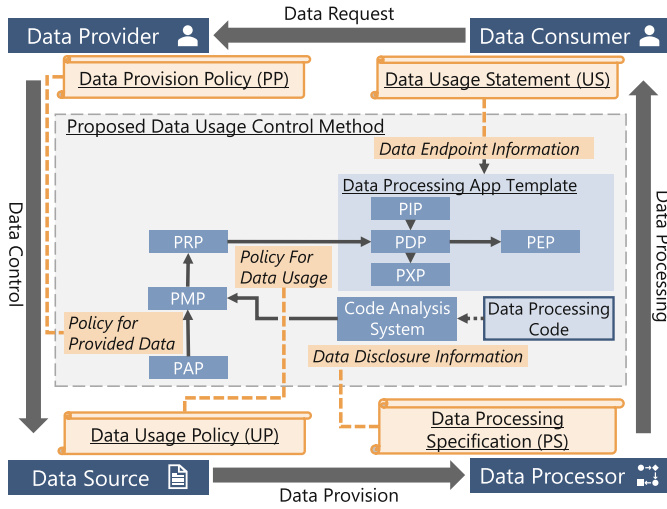


Fig. 5. Data model for proposed data usage control method [25].

for taint analysis. In short, data output through this function may be considered sufficiently anonymized and may be treated as data unrelated to the provided data. We will discuss specific designs in Section V.

V. VDDPI: VERIFIABLE DECENTRALIZED DATA PROCESSING INFRASTRUCTURE

This section describes the data processing infrastructure based on the security requirements defined in Section III.

A. Data Model for Proposed Data Usage Control Method

This section outlines the information held by each entity to realize the usage control proposed method described in Section IV. Fig. 5 shows the relationship between the information and the proposed method. Four types of information are used to manage provided data and data processing code, and to present requests and conditions. Data endpoint information to be input into the data processing app is represented by a *data usage statement* (US). The US contains information about provided data to be used and how the data will be used. It is also used when applying to data providers for data usage (details are given in Section V-D). The policy created by the PAP is described as a *data provision policy* (PP) and contains information about the provided data and the conditions for each provided data. Data disclosure information in data processing is managed as a *data processing specification* (PS) and presented to the PMP. The PMP creates a *data usage policy* (UP) based on this information and provides it to the PDP via the PRP.

In Fig. 6, the relationship among PS, PP, and US is illustrated. First, the PP corresponding to the URI contained in the US is selected, and it is also checked whether the app ID in the US is included in the set of app IDs authorized for use. This check is optional. Then, the PS corresponding to the app ID in the US is identified, and it is verified using the inputs and outputs contained in that PS, whether disclosure is permitted by the PP. Finally, based on the usage counter, location, and

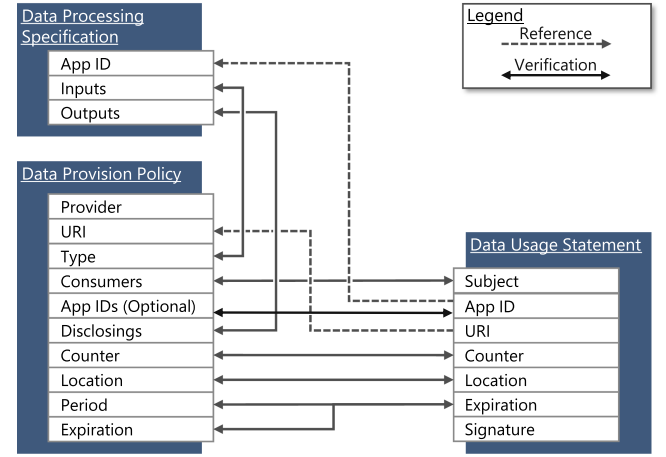


Fig. 6. Relationship between PP, US, and PS.

Data Processing Specification	
App ID	"12345678..."
Inputs	{"person"}
Outputs	{""} (No Output)
Data Provision Policy	
Provider	"provider.example.com"
URI	"https://..."
Type	"person"
Consumers	{"consumer.example.com"}
App IDs	{"12345678..."}
Disclosing	{""} (No Disclosure Permitted)
Counter	5
Location	"JP"
Period	90
Expiration	"2026-12-01"
Data Usage Statement	
Subject	"consumer.example.com"
App ID	"12345678..."
URI	"https://..."
Counter	5
Location	"JP"
Expiration	"2025-10-13"
Signature	"abcdef..."

Fig. 7. Specific examples of PP, US, and PS.

expiration date specified in the US, it is confirmed whether these conditions are satisfied by the PP. If all checks are passed, the data provider authorizes the data provision. Furthermore, based on the authorized conditions, a UP is generated at the time of data provision.

PP is expressed as $PP = (Provider_{PP}, URI_{PP}, Type_{PP}, Consumers_{PP}, AppIDs_{PP}, Disclosings_{PP}, Counter_{PP}, Location_{PP}, Period_{PP}, Expiration_{PP})$. $Consumers_{PP}$, $AppIDs_{PP}$, and $Disclosings_{PP}$ can contain multiple elements and are represented by sets. US is expressed as $US = (Subject_{US}, AppIDs_{US}, URI_{US}, Counter_{US}, Location_{US}, Expiration_{US}, Signature_{US})$. PS is expressed as $PS = (AppID_{PS}, Inputs_{PS}, Outputs_{PS})$. $Inputs_{PS}$ and $Outputs_{PS}$ may be composed of multiple elements.

As an illustrative example, Fig. 7 illustrates specific instances of these policies. In the PP, resources identified by URIs (https://...) are specified, and permits data provision only when the stated conditions are met. With respect to disclosure conditions, only the properties explicitly listed here are allowed to be disclosed; however, this policy does not permit any disclosures. The US represents a request for the resource specified by the URI (https://...) and explicitly defines the conditions under which it

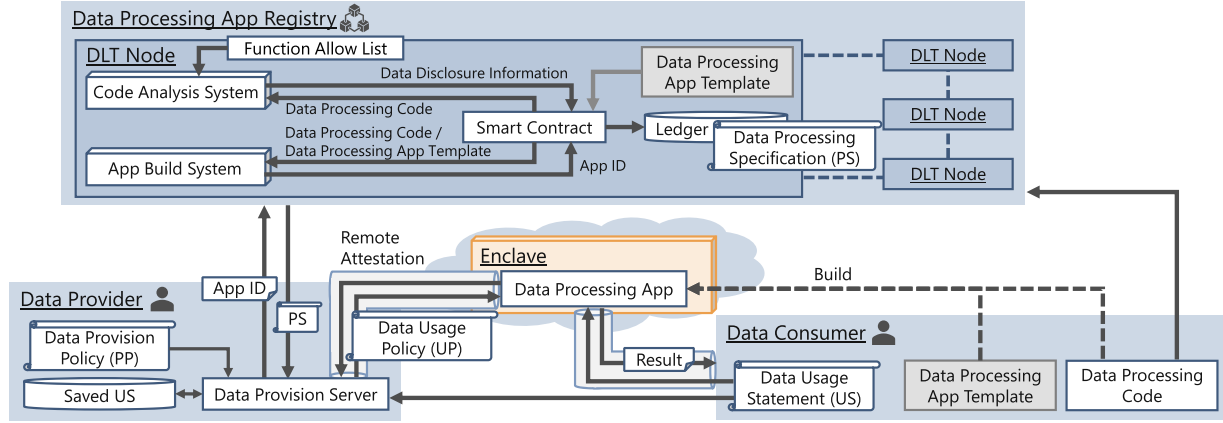


Fig. 8. Overview of VDDPI. (In this figure, the data processing app is running in the cloud, although it can be deployed within the local environment of the data consumer.)

may be used. The PS describes the app's analysis results, where, in this example, this granularity of the information recorded here is at the property level of each argument. When these policies are applied, data provision is authorized because the conditions defined in the PP are satisfied.

B. Overview

Fig. 8 shows an overview of VDDPI. In this infrastructure, data providers (DPs), data consumers (DCs), and a data processing app registry (AR) exist as entities. In VDDPI, the data processing app runs in a TEE enclave, ensuring the integrity of the app and the confidentiality of provided data during processing. Furthermore, DCs send their source codes for the implemented data processing code to the AR, and the analysis of the code uses a smart contract in the permissioned DLT. This analysis could be performed by the data provider; however, it would be an arduous work. In addition, if the data provider performs the analysis, the data consumer needs to provide the processing code to the data provider, so the data consumer may be hesitant to share the processing code and unable to analyze the code if the processing code uses a highly confidential algorithm. To address this, the analysis in VDDPI is performed using permissioned DLT so that the data processing code is not disclosed to anyone other than the authorized entities that comprise the AR. In addition, smart contracts ensure the integrity and fault tolerance of the analysis process.

The data provision server obtains the PS using the ID of the data processing app. To make the PS retrievable using the app ID, it is necessary to link the processing app code to the data processing app disclosure information with the app ID, and record it in distributed ledgers. The app ID is calculated at the build time. To match the ID recorded in the AR with the ID of the app executed by the DC, the infrastructure employs the same environment for app build system and DCs with container technology. MRENCLAVE, used as the app ID, is a hash measurement of the app. The AR calculates the app ID using its own template. Therefore, if a DC modifies the template, the app ID will not match, allowing detection of tampering. Consequently,

VDDPI can ensure compliance with app templates by verifying the app ID.

VDDPI enables the continuous control of provided data, including temporary storage, by employing the data usage control method described in Section IV. In order to guarantee the integrity of UPs and the confidentiality of temporarily stored data on the data consumer's side, TEE sealing is employed for the storage of data. When temporarily stored data is used, the data processing is executed after unsealing provided data and UP, and confirming that the UP is satisfied. In order to achieve continuous control of stored data in data processing apps, UPs are managed in the form of a so-called *sticky policy* [39].

C. Two Types of Control Supported in VDDPI

In VDDPI, conditional control is implemented through event-trigger-based control and usage-count-based control. For event triggers, control is carried out using external trusted information sources. In this paper, we support location- and time-based control. For usage-count-based control, the remaining number of allowed uses is recorded in sealed data, and control is enforced based on this information. Since sealed data is tamper-resistant and the integrity of decrement processing is guaranteed by a TEE, it is ensured that the number of uses does not exceed the limit specified by the data provider. The extension of controllable conditions is discussed in Section IX.

D. Workflow

1) *Phase 1 (App Registration)*: This phase registers the data processing apps (Fig. 9). First, the DC implements the data processing code and sends it to the AR. Upon receiving it, the AR executes the smart contract to obtain a PS. The smart contract analyzes the data processing code received from the DC using the code analysis system. This system verifies the format of the data processing code, including checking called functions, and then static taint analysis is used to obtain the input-output relationship of provided data. Functions that can be called within data processing code are managed by the registry, however, this authorization is performed after manually confirming that no

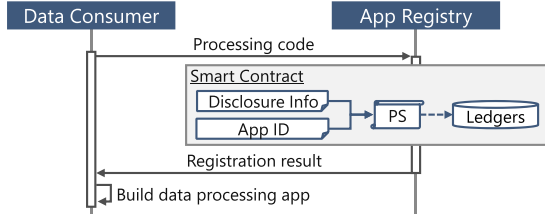


Fig. 9. Sequence diagram of app registration in VDDPI.

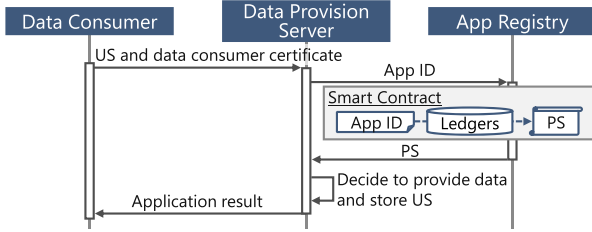


Fig. 10. Sequence diagram of data usage application in VDDPI.

unintended data leaks or unauthorized processing will occur within the function. Then, the app build system generates an app ID for the data processing code. This build system receives data processing code from the DC and the specified app template, and uses this code to perform the build. Next, the smart contract stores the PS that contains the data disclosure information and app ID in ledgers. The AR responds to the DC's browser with the registration result. Finally, data consumers build data processing apps to run within an enclave. At this point, the DC can verify that the registered app is the same as the one built in its own environment by comparing MRENCLAVE.

2) *Phase 2 (Data Usage Application)*: This phase applies for data usage (Fig. 10). The DC obtains a data consumer certificate (CC) from the certificate authority (CA) operated by the AR before applying. The DC sends the US to a data provision server to apply for data usage. After the data provision server authenticates the DC with the CC, the DP queries the AR for the PS using the app ID contained in the US, and the AR responds with the PS obtained using the smart contract. Although the AR is constructed upon a permissioned DLT, which means unauthorized nodes do not participate, nodes may still be compromised. Therefore, the DP must repeat the query for multiple AR nodes to ensure the integrity of the PS.

Next, the data provision server makes data provision decisions based on the obtained CC, PP, US, and PS. In this provision decision, the DP uses the CC to authenticate the DC. The DP then uses the PP to check if the DC and the app are authorized. In addition, the DP checks whether the data will be disclosed using the PS, and decides based on the PP whether to provide data under the conditions of use such as count, location, and duration described in the US presented by the DC. If data provision is permitted, the DP stores the US. Finally, the data provision server notifies the DC of the result of an application for data usage.

This notification of application completion comprises two elements: firstly, a JSON Web Token (JWT) signed by the data

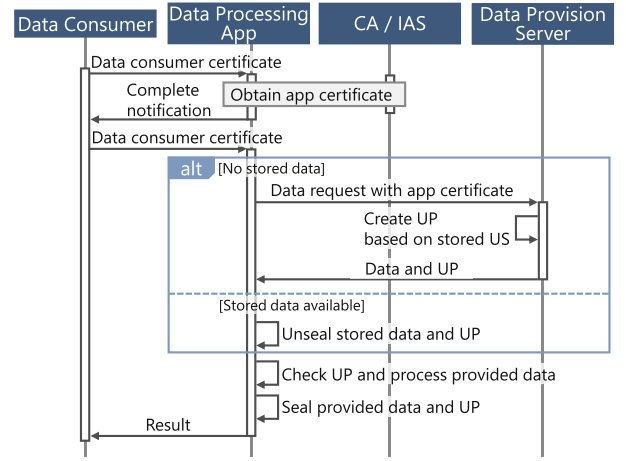


Fig. 11. Sequence diagram of data processing in VDDPI.

provision server; and secondly, a certificate containing the public key required for signature verification. The data processing app utilizes this information to confirm the completion of the data usage application.

3) *Phase 3 (Data Processing)*: This phase processes data (Fig. 11). First, the data consumer sends the CC, and establishes TLS connection with the app. At this time, the app authenticates the data consumer using TLS client authentication. The app obtains an app certificate from the CA and Intel Attestation Service (IAS) using the distinguished name of the CC. This certificate contains the information required for remote attestation, including the app ID.

Subsequently, the data processing app begins execution, but first verifies that the usage application has been completed by validating the JWT obtained in phase 2. Next, the DC's data processing app running in the enclave requests data to the data provision server through remote attestation. The data provision server verifies the data processing app with the app certificate and retrieves the stored US using the app ID included in the certificate. The data provision server creates an UP and sends the data to the app along with it. The data processing app verifies the received UP with respect to the location and time at which the data processing is to be carried out. This verification of the UP enables continuous control of the provided data. Data processing is performed only when the usage conditions are satisfied. The results were output outside the enclave. Finally, the data processing app checks the number of available uses specified by the UP and saves provided data and the UP. When the cached data are available, the data processing app retrieves from the stored data and the UP. The stored data and the UP are encrypted using TEE sealing so that stored data cannot be obtained outside the enclave or from other enclave apps.

VI. SECURITY EVALUATION

A. Requirements Verification

In this section, we evaluate that VDDPI meets the security requirements described in Section III-E.

VDDPI uses remote attestation between the data provision server and the app to authenticate the app when data are provided. This remote attestation is TLS-based (RA-TLS), and data providers can verify data consumers by verifying the X.509 certificate on the server during client authentication. Additionally, by verifying the app's input and output using the PS obtained from the AR, the DP can be confident in the confidentiality of the provided data. However, an attacker could compromise some nodes and replace the PS with a malicious one, potentially misleading the DP into believing the app's confidentiality is ensured. Against this attack, the DP can correctly perform app authentication, including processing specifications, by comparing responses from multiple AR nodes. These authentication satisfy SR-1 and SR-2. In addition, the data processing app employs TLS client authentication for data consumer authentication, which satisfies SR-3.

VDDPI uses TEE sealing to encrypt and store provided data and data usage policies on the data consumer side. As VDDPI performs TEE sealing based on MRENCLAVE, the sealed files can only be accessed by the enclave application that carried out the sealing. Therefore, SR-5, and SR-7 are fulfilled by the use of TEE sealing. SR-4, however, is only partially satisfied by this mechanism, as there remains a risk that the provided data could be rolled back. VDDPI cannot prevent such attacks, as described in Section III-A. Nevertheless, existing studies have leveraged monotonic counters to counter rollback attacks [40], [41], and VDDPI could also mitigate this threat by incorporating a similar mechanism.

The communication channels between the data provision server and the data processing app and between the app and the data consumer are encrypted as TLS is used. Thus, SR-6 and SR-8 are satisfied because these channels are protected by this encryption.

The data processing app runs within an enclave in VDDPI. This protects the memory of the running app from tampering, ensuring SR-9. Apps implemented by data consumers are verified by a smart contract in the registry and linked to an app ID as a PS. The data provision server verifies that the app ID obtained at remote attestation with the app matches the app ID of the PS before providing data, guaranteeing that the app has been verified with the smart contract. This guarantees that the app adheres to the specified template and has not been tampered with. However, data leakage remains possible if memory vulnerabilities in the TEE are exploited, and VDDPI could be subject to such attacks, as discussed in Section III-A. Consequently, SR-10 can only be partially satisfied. Potential countermeasures include the use of memory-safe programming languages and compiler-based protections, but applying these measures to VDDPI is left as future work. Moreover, apps that are implemented according to the template adhere to data usage policies presented. These policies are created by the data provision server based on the data usage application, the data usage control mechanism in the app guarantees compliance with the policies indicated in the US. This enforcement of compliance with the template eliminates any room for repudiation by data consumers, and this data usage control mechanism satisfies SR-11. Finally, the verification of

apps with a smart contract uses static taint analysis to verify whether or not provided data are disclosed in data processing. As mentioned earlier, the PS are guaranteed to be the app with which the data provision server communicates by the integrity of the TEE application and the information stored in distributed ledgers, and by the authentication of the application via remote attestation. Therefore, the data provider can obtain information on data processing of the app before providing data, and SR-12 is satisfied.

B. Formal Verification

In this section, we perform formal verification on the VDDPI protocol and show the security of this protocol. ProVerif is based on a formal model called the Dolev-Yao model [42]. ProVerif can be used to prove properties related to secrecy, authentication, strong secrecy, and equivalence of cryptographic protocols.

We wrote the VDDPI protocol in a ProVerif-specific language for verification with ProVerif. The code is available at <https://github.com/VDDPI/VDDPI/tree/main/proverif/main>. This paper models each process running in VDDPI. In this model, communication channels that are encrypted with TLS on the protocol are assumed to be private. In addition, some of these communication channels manage connections by sending random numbers in order to stop verification by ProVerif. In order to evaluate whether data usage by malicious apps occurs, we explicitly define processes for apps that have not applied for data usage, and verify that provided data are actually being used in these processes. This malicious app models that the data usage controls are not performed and that provided data is disclosed as a result of the process.

In addition, in ProVerif, the verification target must also be modeled. This modeled requirement is called a query. In this study, we assume five attacks as described in Section III-B. Therefore, we convert these five attacks into queries. The results of the conversion are shown in Table V. The queries shown in the table are for the case where no temporarily stored data are used. To verify the data processing with temporarily stored data, it is necessary to replace events *data_processed* and *policy_checked* with *cached_data_processed* and *cached_policy_checked*, respectively. Here, we describe queries when the data processing does not use stored data.

Queries corresponding to UA-1 and UA-2 are verified for non-reachability. These queries verify that an adversary has no access to each of provided data and processing results. On the other hand, UU-1, UU-2, and UU-3 are queries about corresponding assertions. UU-1 is defined with reference to the query used by Arshad et al. [12] in their formal verification of the enforcement of data usage control obligations. This query indicates that when some data was processed with a certain data processing policy, a policy check was previously performed by the data processing app for the same data and policy, and that the data provider created the policy before checking the policy. UU-2 represents that the data provider has previously authorized the data processing app to use the data when it is processed by the

TABLE V
QUERIES WITHOUT TEMPORARILY STORED DATA

Attacks	Queries
UA-1	query attacker(<i>providedData</i>).
UA-2	query attacker(<i>processData(providedData)</i>).
UU-1	query <i>data</i> :bitstring, <i>dc_dn</i> :bitstring, <i>app</i> :bitstring, <i>up</i> :policy; event (<i>data_processed(data, dc_dn, app, up)</i>) $\Rightarrow$ (event (<i>policy_checked(data, up)</i>) $\Rightarrow$ event (<i>policy_created(data, up)</i>)).
UU-2	query <i>data</i> :bitstring, <i>dc_dn</i> :bitstring, <i>app</i> :bitstring, <i>up</i> :policy; event (<i>data_processed(data, dc_dn, app, up)</i>) $\Rightarrow$ event (<i>DC_authorized(data, dc_dn)</i>).
UU-3	query <i>data</i> :bitstring, <i>dc_dn</i> :bitstring, <i>app</i> :bitstring, <i>up</i> :policy; event (<i>data_processed(data, dc_dn, app, up)</i>) $\Rightarrow$ event (<i>app_authorized(data, app)</i>).

TABLE VI
EXPERIMENTAL ENVIRONMENT OF EACH ENTITY

Entity	Azure VM Size	Processor	EPC / Enclave Size	vCPUs	Memory
Data Consumer	Standard DC1ds v3	Intel Xeon Platinum 8370C @ 2.80GHz	4 GB / 1 GB	1	8 GB
Data Provider	Standard D2s v6	Intel Xeon Platinum 8573B	N/A (no SGX)	2	8 GB
App Registry	Standard D4ls v6	Intel Xeon Platinum 8573B	N/A (no SGX)	4	8 GB

User Code (Python Code)
VDDPI (Python Code): ≥ 606 LOC
Python Interpreter (Modified): 858 KLOC
Gramine-SGX: 1,348 KLOC

Fig. 12. TCB size for each program running on the data consumer side.

app. UU-3 replaces the part of the query in UU-2 that represents the app with the data consumer.

We performed verification using ProVerif version 2.05 on the model formalizing the aforementioned VDDPI protocol and the query formalizing the attacks. The results show that all queries are satisfied with or without temporarily stored data. From the above, the assumed attacks will not occur in the VDDPI protocol, indicating that it is secure.

VII. IMPLEMENTATION AND PERFORMANCE EVALUATION

A. Implementation

We implemented a prototype of VDDPI using SGX as a TEE and Gramine-SGX [43] as a data processing app execution environment. Gramine-SGX, a library OS, protects the entire process with enclaves. Hyperledger Fabric version 2.4 was used as the DLT for an AR. For data flow analysis, we employed Pysa [44], a static taint analysis tool developed by Meta. The prototype ran in the environment shown in Table VI. Intel SGX SDK version 2.26.100.0 was used for the DC. The data processing app was implemented in CPython version 3.6.9 on the Gramine-SGX framework version 1.9. Source code and documentation are available at <https://github.com/VDDPI/VDDPI>.

The programs that run on the data consumer side and each TCB size are shown in Fig. 12. The TCB size of Gramine-SGX, the enclave execution environment, is 1,348 KLOC (Kilo Lines Of Code), and the python interpreter running on it is 858 KLOC.

Furthermore, although there may be some differences depending on the code of the permitted functions, the TCB size of the data processing app template provided by VDDPI is approximately 606 LOC (Lines Of Code).

B. Experimental Setup

In the performance evaluation scenario, there is one DC and AR consists of three organizations, with one DP in the data usage application and changing the number of DPs in the data processing phase. The DC, DPs, AR, and API servers for data usage control and remote attestation were run in the same local area network (LAN). In addition, three Hyperledger Fabric peer nodes used by the AR were run in the same machine.

We used approximately 100 bytes of randomly generated test data as the data provided by each DP to evaluate performance. This test data represents the personal data of individuals in the use cases of Section II-A. In this evaluation, the DC obtains the personal data and derives generalized attributes: the age is converted into an age group, the location information is converted into floor information, and the disability information is converted into a disability category (e.g., difficulty in mobility or difficulty in obtaining evacuation information). The detailed raw data of individuals are not disclosed.

C. Performance Evaluation of VDDPI Setup

We measured a performance in the app registration phase and the data usage application phase. The data processing app source code of approximately 5 KB was used to register the app. The execution time of the subsystems in each phase was measured. The execution time is presented in Figs. 13 and 14. In this measurement, the app registration phase took approximately $12,302 \pm 844$ ms and the data usage application phase took approximately $6,865 \pm 43$ ms.

D. Performance Evaluation of Data Processing

To evaluate scalability and processing performance, the number of DPs, each corresponding to one individual, was increased from 1 to 1,000. Fig. 15 shows the processing time and memory

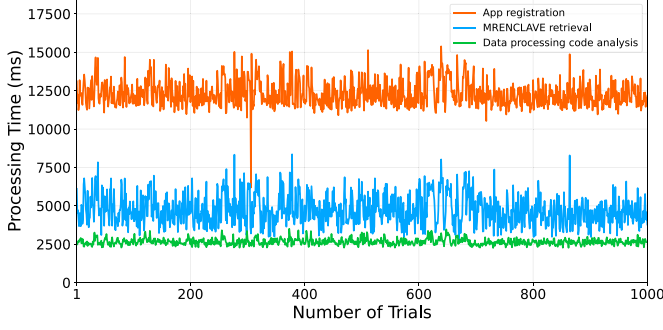


Fig. 13. Execution time for app registration phase.

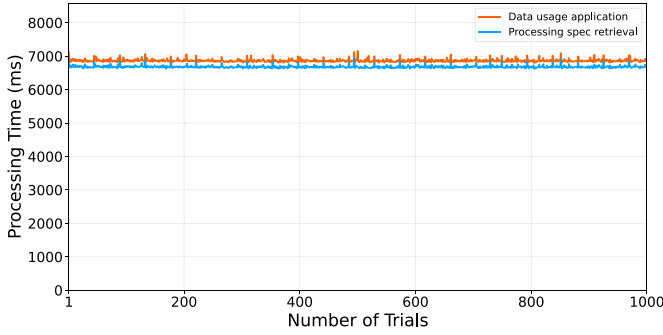


Fig. 14. Execution time for data usage application phase.

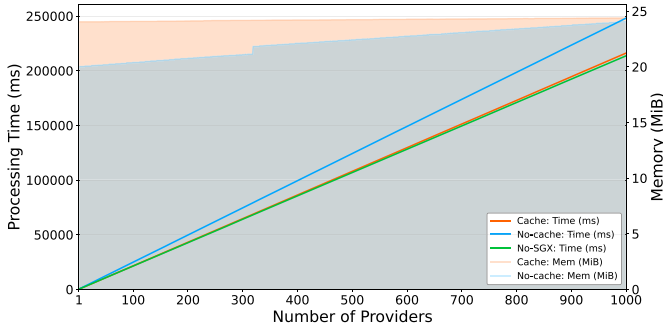


Fig. 15. Execution time for data processing phase in TEE and REE.

usage for three configurations: (1) with personal data cached (blue line), (2) without caching (orange line), and (3) without caching and without using SGX (green line). Configurations (1) and (2) were executed in the TEE using SGX, whereas configuration (3) was executed in the Rich Execution Environment (REE) without SGX. As expected, the processing time of configuration (1) was shorter than that of configuration (2) due to the effect of caching. Comparing configurations (2) and (3), the absence of SGX noticeably reduced the processing time — about 250 seconds and 210 seconds for 1,000 DPs, respectively — indicating that the SGX overhead is moderate but not negligible. Memory usage increased almost linearly with the number of DPs in all configurations. The cached configuration consumed slightly more memory due to data storage in the cache, while the difference between the cached and non-cached configurations remained small. The maximum memory consumption observed

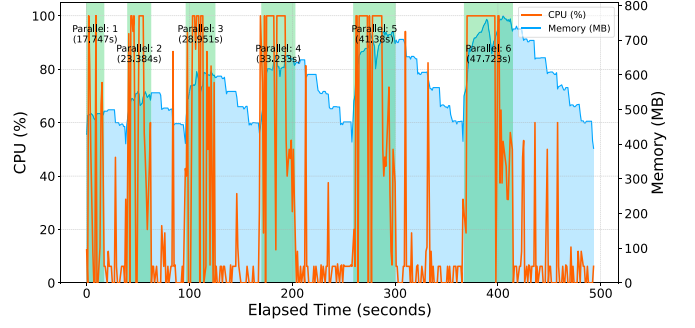


Fig. 16. Runtime, memory usage, and CPU utilization with varying numbers of data consumer nodes.

was approximately 25 MB, even when the number of data providers reached 1,000, where each provider's data size was about 100 bytes. These results indicate that VDDPI introduces only a small and predictable memory overhead, suggesting that it is practical for deployment in real-world cloud-based environments where TEE-enabled virtual machines are available.

E. Scalability Evaluation with Varying Numbers of Data Consumers

To evaluate the scalability of VDDPI, we conducted an additional experiment focusing on the data processing phase. The experimental configuration was the same as that described in Section VII-B, except that the number of DC nodes was varied to examine the effect of parallel execution. Specifically, the number of DC nodes was configured as $1 + N$ ($N = 1-6$), where N represents the degree of parallelism. Each DC node executed identical data processing tasks in parallel within the same machine.

Fig. 16 shows the runtime, memory usage, and CPU utilization during the data processing phase with varying numbers of DC nodes. As the degree of parallelism increased from 1 to 6, the total processing time grew from approximately 17.7 seconds to 47.7 seconds, showing an almost linear trend. The CPU utilization approached 100% during each processing period, indicating that the available computing resources were fully engaged by the concurrent DC nodes. This high utilization occurred because the processing tasks were continuously executed without idle time, and no apparent CPU bottlenecks or excessive contention were observed. Memory usage also increased moderately as the number of parallel tasks increased, reflecting the additional overhead required for concurrent enclave executions.

F. Performance Comparison of VDDPI and PrivacyGuard

We compared VDDPI performance with PrivacyGuard [17]. We used programs from a repository (<https://github.com/yangsec/PrivacyGuard>) to measure execution time of PrivacyGuard. The data processing performed by VDDPI and PrivacyGuard is the training of the SVM classifier. LIBSVM [45] was employed for this data processing, and the adult dataset from the UCI Machine Learning Repository [46] was used to simulate the data

TABLE VII
COMPARISON OF RELATED WORK

Related Work	General		Data Usage Control		Flexibility of Processing		Processing Code Security	
	Technology	Data Type	Conditions Enforcement	Data Processing	Programmable	Implementer	Confidentiality	Code Verification
OB-ABE [12]	TEE, ABE	Healthcare	Y	N	-	-	-	-
ReGov [13]	TEE, DLT	Personal	Y	N	-	-	-	-
DUCE [14]	TEE, DLT	IoT	Y	N	-	-	-	-
Regueiro et al. [15]	HE, DLT	General	N	Y	Partially	Consumer	Partially	Participant (Provider)
More et al. [19]	-	Personal	N	Y	Y	N/A	N	Seller (Provider)
b-DTC [20]	TEE, DLT	Healthcare	N	Y	Y	Operator	N	User (Provider)
PDO [16]	TEE, DLT	General	N	Y	Y	Consumer	Partially	User (Provider)
DNAT [10]	TEE, DLT	Any	N	Y	Y	Consumer	Partially	Owner (Provider) / Trusted Third Party
PrivacyGuard [17]	TEE, DLT	Personal	N	Y	Y	Pre-defined	Partially	Trusted Third Party
TrustControl [18]	TEE	Any	N	Y	Y	Consumer	Partially	Attestation Server
MECT [47]	TEE	Any	N	Y	Y	Consumer	Y	Provider's Server
Lohmöller et al. [21]	TEE	General	Y	Y	Y	N/A	N	Owner (Provider)
VDDPI	TEE, DLT	Personal	Y	Y	Y	Consumer	Partially	Smart Contract

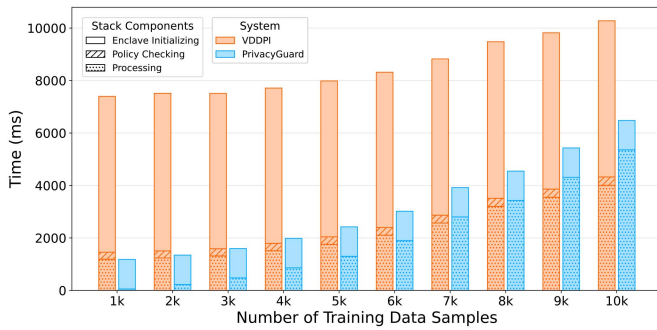


Fig. 17. Runtime comparison with PrivacyGuard.

source. The number of samples was measured from 1 k to 10 k, and we calculated the average of 10 runs for each size.

Fig. 17 shows the result of the comparison. Data loading and result encryption required only minimal time, so they were aggregated under “Data Processing”. As shown in figure, when the number of training data samples is small (e.g., 1 k), the processing time of VDDPI is higher than that of PrivacyGuard, with a difference of approximately 1,135 ms. However, as the training dataset increases, this difference gradually decreases and reverses at around 7 k samples, where VDDPI becomes faster. At 10 k samples, VDDPI outperforms PrivacyGuard by approximately 1,350 ms, indicating that the relative efficiency of VDDPI improves as the data size grows. The crossover around 7 k samples indicates that VDDPI’s higher initialization cost is gradually outweighed by its more efficient per-sample processing. However, a more detailed analysis is required to clarify the underlying causes of this performance shift, such as the impact of enclave memory management and I/O overhead. In contrast, both Enclave Initialization and Policy Checking remain almost constant regardless of the number of training samples. The initialization time is approximately 5,947 ms for VDDPI and 1,121 ms for PrivacyGuard, while Policy Checking, which is performed only in VDDPI, takes about 290 ms. The difference of the initialization time is caused by the implementation of VDDPI using Gramine-SGX and could be significantly improved by changing the software employed for the implementation.

VIII. RELATED WORK

In this section, we present related work such as data processing using a TEE and DLT. Table VII shows the result of comparison of related work. In addition to the used technology and the assumed data type, the comparison focuses on data usage control, security of the data processing code, and flexibility of data processing. In the context of data usage control, we conducted an analysis of the existence of a mechanism that complies with the presented obligations and conditions, and the ability of data providers to specify data processing. In addition, we examine the flexibility of data processing by evaluating the programmability of the code and comparing the entities that implement data processing. In data processing code security, we compare the confidentiality of the code itself and the entities by which the data processing code is verified.

As a study focusing on the enforcement of obligations and conditions in data usage control, OB-ABE [12] extends classical attribute-based encryption (ABE) with enforceable obligations using a TEE. OB-ABE allows data providers to define obligations, thereby giving data providers flexibility in controlling provided data. Moreover, OB-ABE has been proven secure through formal verification by ProVerif. ReGov [13] is a decentralized data usage control architecture using TEE and DLT. ReGov extends the existing Solid access control mechanism to enable data usage control. DUCE [14] is a trusted and distributed enforcement architecture using TEE and DLT to enforce usage control policy. These studies have mechanisms to enforce obligations and conditions for data usage control; however, they have not performed the verification of data processing at the data destination.

As a study considering the verification of data processing, Regueiro et al. [15] proposed a privacy-enhancing distributed protocol for data aggregation using blockchain and homomorphic encryption (HE). In this protocol, participants process a query presented by a data consumer, and an operator representing the participants aggregates the results and returns them to the data consumer. More et al. [19] proposed a private data market architecture. This architecture allows flexible access control based on the policy system, and the computation system in this architecture validates policies based on the attributes and

roles of the data buyer at the time of data purchase. b-DTC [20] is a blockchain-based platform for decentralized trusted computing. b-DTC performs computations off-chain, using TEE, and records the performance and other information of the computation as reputation on the blockchain via a smart contract. Private Data Objects (PDOs) project [16] provides technology for execution of off-chain smart contracts with confidentiality using SGX. PDOs, like VDDPI, focus on avoiding a single point of failure. It is expected that the contract used for data processing is shared offline in order to agree on its use prior to use in PDOs. DNAT (Data aNd Application Tracking) [10] proposes a platform for traceability by combining blockchain and a TEE (Intel SGX) to record and track application executions and data usage in a tamper-resistant manner. When the data is sensitive, DNAT assumes that pre-verification has been performed to ensure that the application is non-malicious, performed by the data owner or by a trusted third party with access to the source code. DNAT then executes the application over the data inside an SGX enclave and releases the decryption key only after attestation, ensuring that the pre-verified code is executed without being tampered with. PrivacyGuard [17] enables an individual's control over the usage of private data using TEE and blockchain. In PrivacyGuard, data usage contract is recorded on the blockchain, data processing is performed in off-chain contract execution environment provided by a TEE. The data processing in PrivacyGuard is assumed to be ratified by trusted sources in advance, and the verification of data processing is not performed in the system. These studies consider the verification of the data processing. However, it is assumed that the verification is conducted by data providers or trusted third parties outside the framework. In addition, the confidentiality of the data processing code is compromised when the code is made publicly available on data markets.

As a study that takes into account the verification of the data processing implemented by data consumers, TrustControl [18] is a data usage control system focusing on protecting the sensitive data and function code while preventing data misuse and leakage. Remote attestation in TrustControl is designed to enable to verify the function codes implemented by data consumers. However, this verification is performed on a single attestation server, thereby the server can be a single point of failure of the verification. MECT [47] is a SGX-based architecture for trusted remote policy enforcement. In MECT, the data consumer implements the app according to a predefined template, and the data provider's server verifies the code. As the number of data providers increases, the cost of verification is anticipated to rise, given that each provider must verify the code. This differs from VDDPI, in which analysis information is managed in the data processing app registry, and the data provider acquires the information from the registry.

As a study that considers both the enforcement of obligations and conditions in data usage control and the verification of data processing, Lohmöller [21] proposed a federated data platform architecture using TEE and DLT for reliable enforcement of complex policies. This architecture uses TEE to guarantee the enforcement of obligations and conditions for data usage control when providing data to untrusted entities. Although this study

technically guarantees the enforcement of conditions for data usage control and the control of data processing content, code verification is not performed mechanically. As a result, when the size of code to be verified is considerable, it becomes impractical to perform the verification [23]. VDDPI can handle such cases, as the smart contract verifies data processing and enables data providers to make data provision decisions based on the app specifications. In addition, this verification in VDDPI is performed with permissioned DLT, and only data disclosure information is disclosed to data providers. This approach partially ensures the confidentiality of the data processing code.

IX. DISCUSSION

A. Confidentiality Challenges of Data Processing Code

As discussed in Section III-A, in VDDPI, the registry can inspect the data processing code implemented by data consumers. Because this registry is built on a permissioned DLT, access is controlled rather than fully open. However, if the data processing logic must remain confidential, VDDPI is not an appropriate choice. We therefore consider VDDPI suitable for scenarios where confidentiality is not required, such as general route search algorithms.

For apps where the source code cannot be disclosed, applying VDDPI remains challenging. To address this limitation, one possible approach is to implement the data processing app to perform dynamic taint analysis on its own execution and to run only when the data usage policies are satisfied, thereby avoiding the need to disclose the code. Nevertheless, this approach raises concerns about increased execution time due to the overhead of runtime taint tracking. These points remain challenges for the future.

Another possible approach is to execute the AR-side workflow, specifically the smart-contract-driven analysis and build steps, inside an attested TEE environment, so that proprietary processing code can remain confidential even at registration time. For instance, confidential smart-contract frameworks such as Hyperledger Fabric Private Chaincode (FPC) [48] execute chaincode within Intel SGX enclaves, which can protect sensitive inputs from peer operators. However, VDDPI's registration workflow not only analyzes the submitted code but also builds the data processing app and derives its app identifier (MRENCLAVE). Conducting this compilation inside such an enclave is technically challenging and likely impractical in many deployment settings, and therefore confidentiality during compilation remains an open issue. Moreover, extending TEE usage to the registration toolchain introduces a bootstrapping issue: the analysis and compilation components become part of the trusted computing base and must be trusted for both correctness and non-leakage.

B. Towards Flexible Data Usage Control

Data usage policy scope is an important requirement for data usage control solutions [49]. VDDPI currently supports only a limited set of basic conditions and cannot enforce controls based on diverse conditions. In contrast, IDS defines 21 types

of policy classes as policy templates for usage control [38], and it is desirable for VDDPI to also support the implementation of such policy classes. As a possible approach to realize this, we are considering a method in which the registry manages the control processes executed within data processing apps on the distributed ledger it maintains. This would enable VDDPI to achieve flexible control and monitoring in accordance with various data provision scenarios.

C. Challenges in Adapting Existing Applications to VDDPI

As described in Section III-A, adapting existing applications to VDDPI involves several challenges. In conventional data processing apps, numerous functions are typically invoked; however, VDDPI imposes restrictions on function calls, which prevents such applications from running as-is. Consequently, while developing new applications from scratch would not pose major difficulties, porting existing applications can be considerably labor-intensive. In addition, data providers must define appropriate data provision policies. These efforts can be substantial for applications with extensive dependencies or frequent function calls, potentially leading to significant redevelopment costs.

Regarding the automation of adaptation, its feasibility would depend on a mechanism capable of automatically converting functions and system calls into those allowed by VDDPI. Nevertheless, we consider this task to be challenging in practice.

D. Limitations of the Current Implementation

In the current implementation of VDDPI, we employ Gramine-SGX and the CPython interpreter. Since both of these components have a relatively large LoC, they broaden the potential attack surface. In particular, Gramine-SGX may exhibit unintended behavior due to vulnerabilities in its dependent libraries or exception handling, as suggested by the trends in its security patches. Consequently, VDDPI can be also exposed to such latent security risks. Implementing VDDPI with Intel SGX SDK and C is expected to reduce the attack surface, and this remains a direction for future work.

E. Trust Model in VDDPI Architecture

While VDDPI employs TEE to protect data processing, the system architecture includes components that operate outside the TEE boundary and require explicit trust assumptions, as described in Section III-A. However, the system design manages these trust dependencies to avoid single points of failure in security-critical functions.

The AR's distributed ledger infrastructure, CAs, static taint analysis tools, and build systems operate in conventional computing environments. Critically, the static taint analysis and build processes are invoked through smart contracts executed across multiple AR nodes. This distributed execution means that if any individual node attempts to provide fraudulent analysis results or tampered build outputs, the smart contract execution will be interrupted due to consensus failure. Therefore, the core verification processes that determine data processing safety do

not rely on a single trusted point but rather on the collective integrity of the distributed ledger network.

TEE protection is applied specifically to the most sensitive operations—the actual processing of personal data—while the verification infrastructure leverages blockchain consensus to eliminate single points of trust in the analysis pipeline. Data providers retain ultimate control over provisioning decisions based on these collectively verified results, ensuring that the overall system maintains strong security properties despite the hybrid trust model. This approach demonstrates how confidential computing can be effectively combined with distributed trust mechanisms to achieve both data confidentiality and verification integrity in decentralized data processing environments.

F. Practical Deployment Potential

Based on the experimental results presented in Section VII-D, VDDPI introduces only a predictable amount of memory overhead. However, the data processing time increases linearly with the volume of data. Therefore, for use cases requiring large-scale data processing and quick responses, improving processing efficiency remains a key challenge. In the assumed disaster evacuation support scenario, VDDPI can provide timely assistance in small-scale facilities while maintaining real-time performance. In contrast, when applied to large-scale scenarios such as entire cities, data processing may become a bottleneck, potentially causing delays in delivering assistance. Nevertheless, the performance of VDDPI is largely influenced by the underlying software configuration. It is therefore expected that replacing the software components used in VDDPI could lead to further performance improvements. Consequently, optimizing the implementation for service integration and real-world deployment remains an important direction for future work.

X. CONCLUSION

We proposed a data usage control method with data processing verification, but its practical implementation faces various threats. In this paper, we present VDDPI, a decentralized data processing infrastructure. VDDPI ensures data confidentiality by processing personal data in a TEE enclave and enforces agreed usage conditions. It uses DLT for source code verification, allowing data providers to verify the usage control and data processing before providing data, without relying on a single point of failure. VDDPI allows data processing apps implemented by data consumers to be verified within the infrastructure, thus addressing not only scenarios where apps are pre-implemented and fixed, but also scenarios where there is a need to establish a dynamic trust relationship between data providers and consumers. VDDPI realizes a secure and dependable infrastructure using TEE and blockchain technology, and is expected to promote the utilization of personal data.

We employed STRIDE to the assumed decentralized data processing model to identify threats and verified that VDDPI meets the requirements we derived. We also used ProVerif to evaluate potential attacks and demonstrate VDDPI's safety. A

prototype of VDDPI was implemented and published on Github. Performance evaluation showed that app registration and data usage applications had constant execution times regardless of attempts. However, execution time increased with more data providers, highlighting potential scalability issues.

As future work, we intend to investigate how VDDPI can be applied to other TEEs, such as ARM TrustZone [50]. Furthermore, we will consider how the design described in this paper can be generalized into a broader concept, enabling its applicability across different TEE platforms. In particular, the availability of remote attestation and sealing serves as a key criterion, as these features are essential for determining whether a given TEE can support VDDPI.

REFERENCES

- [1] B. d. A. Almeida et al., "Personal data usage and privacy considerations in the COVID-19 global pandemic," *Ciencia Saude Coletiva*, vol. 25, pp. 2487–2492, 2020.
- [2] Y. Wang, L. Kung, W. Y. C. Wang, and C. G. Cegielski, "An integrated Big Data analytics-enabled transformation model: Application to health care," *Informatio Manage.*, vol. 55, no. 1, pp. 64–79, 2018.
- [3] H. Ma, R. Zhang, G. Yang, Z. Song, K. He, and Y. Xiao, "Efficient fine-grained data sharing mechanism for electronic medical record systems with mobile devices," *IEEE Trans. Dependable Secure Comput.*, vol. 17, no. 5, pp. 1026–1038, Sep./Oct. 2020.
- [4] M. Fernández, J. Jaimunk, and B. Thuraisingham, "A privacy-preserving architecture and data-sharing model for cloud-IoT applications," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 4, pp. 3495–3507, Jul./Aug. 2023.
- [5] C. Ge, W. Susilo, Z. Liu, J. Baek, X. Luo, and L. Fang, "Attribute-based proxy re-encryption with direct revocation mechanism for data sharing in clouds," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 2, pp. 949–960, Mar./Apr. 2024.
- [6] M. Nanni et al., "Give more data, awareness and control to individual citizens, and they will help covid-19 containment," *Ethics Inf. Technol.*, vol. 23, no. 1, pp. 1–6, 2021.
- [7] V. Shubina, S. Holcer, M. Gould, and E. S. Lohan, "Survey of decentralized solutions with mobile devices for user location tracking, proximity detection, and contact tracing in the COVID-19 era," *Data*, vol. 5, no. 4, 2020, Art. no. 87.
- [8] P. Hummel, M. Braun, M. Tretter, and P. Dabrock, "Data sovereignty: A review," *Big Data Soc.*, vol. 8, no. 1, 2021, Art. no. 205395172098201.
- [9] K. U. Fallatah, M. Barhamgi, and C. Perera, "Personal data stores (PDS): A review," *Sensors*, vol. 23, no. 3, 2023, Art. no. 1477.
- [10] J. R. Nascimento Jr, J. B. Nunes, E. L. Falcão, L. Sampaio, and A. Brito, "On the tracking of sensitive data and confidential executions," in *Proc. 14th ACM Int. Conf. Distrib. Event-Based Syst.*, 2020, pp. 51–60.
- [11] J. Park and R. Sandhu, "The ucon_{abc} usage control model," *ACM Trans. Inf. System Secur.*, vol. 7, no. 1, pp. 128–174, 2004.
- [12] H. Arshad, P. Picazo-Sanchez, C. Johansen, and G. Schneider, "Attribute-based encryption with enforceable obligations," *J. Cryptographic Eng.*, vol. 13, no. 3, pp. 343–371, 2023.
- [13] D. Basile, C. Di Ciccio, V. Goretti, and S. Kirrane, "Blockchain based resource governance for decentralized web environments," *Front. Blockchain*, vol. 6, 2023, Art. no. 1141909.
- [14] N. Shi, B. Tang, R. Sandhu, and Q. Li, "DUCE: Distributed usage control enforcement for private data sharing in Internet of Things," in *Proc. Data Appl. Secur. Privacy*, 2021, pp. 278–290.
- [15] C. Regueiro, I. N. Seco, S. de Diego, O. Lage, and L. Etxebarria, "Privacy-enhancing distributed protocol for data aggregation based on blockchain and homomorphic encryption," *Inf. Process. Manage.*, vol. 58, no. 6, 2021, Art. no. 102745.
- [16] M. Bowman, A. Miele, M. Steiner, and B. Valava, "Private data objects: An overview," 2018, *arXiv:1807.05686*.
- [17] Y. Xiao, N. Zhang, J. Li, W. Lou, and Y. T. Hou, "PrivacyGuard: Enforcing private data usage control with blockchain and attested off-chain contract execution," in *Proc. Comput. Secur.*, 2020, pp. 610–629.
- [18] H. Lei et al., "TrustControl: Trusted private data usage control based on security enhanced trustzone," *Comput., Mater. Continua*, vol. 73, no. 3, pp. 5687–5702, 2022.
- [19] S. More and L. Alber, "You shall not compute on my data: Access policies for privacy-preserving data marketplaces and an implementation for a distributed market using MPC," in *Proc. 17th Int. Conf. Availability, Rel. Secur.*, 2022, Art. no. 137.
- [20] Y. Liang, Y. Li, and B.-S. Shin, "A blockchain-based platform for decentralized trusted computing," *Peer-to-Peer Netw. Appl.*, vol. 17, no. 3, pp. 1499–1513, 2024.
- [21] J. Lohmöller, R. Matzutt, J. Loos, E. Vlad, J. Pennekamp, and K. Wehrle, "Complementing organizational security in data ecosystems with technical guarantees," in *Proc. 1st Conf. Building Secure Empowered Cyberspace*, 2024, pp. 49–56.
- [22] M. J. Ryan, "Secret algorithms, IP rights, and the public interest," *Nevada Law J.*, vol. 21, pp. 61–116, 2020.
- [23] D. Natarajan, A. Loveless, W. Dai, and R. Dreslinski, "Chex-Mix: Combining homomorphic encryption with trusted execution environments for oblivious inference in the cloud," in *Proc. IEEE 8th Eur. Symp. Secur. Privacy*, 2023, pp. 73–91.
- [24] A. Avizienis, J.-C. Laprie, B. Randell, and C. Landwehr, "Basic concepts and taxonomy of dependable and secure computing," *IEEE Trans. Dependable Secure Comput.*, vol. 1, no. 1, pp. 11–33, Jan.–Mar. 2004.
- [25] S. Tokuda, S. Kakei, Y. Shiraishi, and S. Saito, "Decentralized data usage control with confidential data processing on trusted execution environment and distributed ledger technology," in *Proc. Netw. System Secur.*, 2025, pp. 127–144.
- [26] H. Janssen, J. Cobbe, C. Norval, and J. Singh, "Decentralized data processing: Personal data stores and the GDPR," *Int. Data Privacy Law*, vol. 10, no. 4, pp. 356–384, 2020.
- [27] S. Wheatley, T. Maillart, and D. Sornette, "The extreme risk of personal data breaches and the erosion of privacy," *Eur. Phys. J. B*, vol. 89, no. 1, pp. 1–12, 2016.
- [28] J. Marti, A. Queral, D. Gasull, and T. Cortes, "Living objects: Towards flexible Big Data sharing," *J. Comput. Sci. Technol.*, vol. 13, no. 02, pp. 56–63, 2013.
- [29] J. Lohmöller et al., "The unresolved need for dependable guarantees on security, sovereignty, and trust in data ecosystems," *Data Knowl. Eng.*, vol. 151, no. C, 2024, Art. no. 102301.
- [30] M. Sabt, M. Achemlal, and A. Bouabdallah, "Trusted execution environment: What it is, and what it is not," in *Proc. IEEE Trustcom/BigDataSE/ISPA*, 2015, pp. 57–64.
- [31] F. McKeen et al., "Innovative instructions and software model for isolated execution," in *Proc. 2nd Int. Workshop Hardware Architectural Support Secur. Privacy*, no. 10, 2013, pp. 1–8.
- [32] I. Anati, S. Gueron, S. Johnson, and V. Scarlata, "Innovative technology for CPU based attestation and sealing," Intel Corporation, White Paper, 2013. [Online]. Available: <https://www.intel.com/content/dam/develop/external/us/en/documents/hasp-2013-innovative-technology-for-attestation-and-sealing-413939.pdf>
- [33] T. Knauth, M. Steiner, S. Chakrabarti, L. Lei, C. Xing, and M. Vij, "Integrating remote attestation with transport layer security," 2019, *arxiv:1801.05863*.
- [34] E. Androulaki et al., "Hyperledger fabric: A distributed operating system for permissioned blockchains," in *Proc. 13th EuroSys Conf.*, 2018, no. 30, pp. 1–15.
- [35] M. Howard and S. Lipner, *The Security Development Lifecycle: SDL: A Process for Developing Demonstrably More Secure Software*. Redmond, WA, USA: Microsoft Press, 2006.
- [36] A. Shostack, *Threat Modeling: Designing for Security*. Hoboken, NJ, USA: Wiley, 2014.
- [37] OASIS Standard, "Extensible access control markup language (XACML) version 3.0," 2013. Accessed: Sep. 14, 2024. [Online]. Available: <https://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html>
- [38] A. Eitel et al., "Usage control in the international data spaces," Aufl., IDS Association, Version 3.0, Berlin, 2021. [Online]. Available: https://internationaldataspaces.org/wpcontent/uploads/dlm_uploads/IDSA-Position-Paper-Usage-Control-in-the-IDS-V3-.pdf
- [39] S. Pearson and M. Casassa-Mont, "Sticky policies: An approach for managing privacy across multiple parties," *Computer*, vol. 44, no. 9, pp. 60–68, 2011.
- [40] A. P. P. Hartono, A. Brito, and C. Fetzer, "CRISP: Confidentiality, rollback, and integrity storage protection for confidential cloud-native computing," in *Proc. IEEE 17th Int. Conf. Cloud Comput.*, 2024, pp. 141–152.

- [41] S. Matetic et al., "ROTE: Rollback protection for trusted execution," in *Proc. 26th USENIX Secur. Symp.*, 2017, pp. 1289–1306.
- [42] D. Dolev and A. Yao, "On the security of public key protocols," *IEEE Trans. Inf. Theory*, vol. TIT-29, no. 2, pp. 198–208, Mar. 1983.
- [43] C.-C. Tsai, D. E. Porter, and M. Vij, "Graphene-SGX: A practical library os for unmodified applications on SGX," in *Proc. 2017 USENIX Annu. Tech. Conf.*, 2017, pp. 645–658.
- [44] Meta, "Pyre - pyre," 2018, Accessed: Sep. 14, 2024. [Online]. Available: <https://pyre-check.org/>
- [45] C.-C. Chang and C.-J. Lin, "LIBSVM: A library for support vector machines," *ACM Trans. Intell. Syst. Technol.*, vol. 2, no. 3, pp. 1–27, 2011.
- [46] B. Becker and R. Kohavi, "Adult," UCI Machine Learning Repository, 1996, doi: [10.24432/C5XW20](https://doi.org/10.24432/C5XW20).
- [47] H. M. Z. Felde, M. Morbitzer, and J. Schütte, "Securing remote policy enforcement by a multi-enclave based attestation architecture," in *Proc. IEEE 19th Int. Conf. Embedded Ubiquitous Comput.*, 2021, pp. 102–108.
- [48] M. Brandenburger, C. Cachin, R. Kapitza, and A. Sorniotti, "Blockchain and trusted computing: Problems, pitfalls, and a solution for hyperledger fabric," 2018, *arXiv:1805.08541*.
- [49] G. Gil, A. Arnaiz, F. J. Diez, and M. V. Higuero, "Evaluation methodology for distributed data usage control solutions," in *Proc. 2020 Glob. Internet Things Summit*, 2020, pp. 1–6.
- [50] T. Alves and D. Felton, "Trustzone: Integrated hardware and software security," *Tech In-Depth*, vol. 3, no. 4, pp. 18–24, 2004.



Shota Tokuda received the BE and ME degrees from the Nagoya Institute of Technology, Japan, in 2023 and 2025, respectively. He is currently a researcher with Hitachi, Ltd., where he is engaged in research on service computing. His research interests include service computing, authentication, access control, and information security.



Shohei Kakei received the BE and ME degrees from Gifu University, Japan, in 2011 and 2013, respectively, and the PhD degree from Kobe University, Japan, in 2019. He has been an assistant professor with the Department of Computer Science, Nagoya Institute of Technology, Japan, since 2019. His current research interests include authentication, trust management, trusted computing, and information security. He is a member of IEICE and IPSJ.



Yoshiaki Shiraishi (Member, IEEE) received the BE and ME degrees from Ehime University, Japan, in 1995 and 1997, respectively, and the PhD degree from the University of Tokushima, Japan, in 2000. From 2002 to 2006, he was a lecturer with the Department of Informatics, Kindai University, Japan. From 2006 to 2013, he was an associate professor with the Department of Computer Science and Engineering, Nagoya Institute of Technology, Japan. From 2013 to 2024, he was an associate professor with the Department of Electrical and Electronic Engineering, Kobe University, Japan. He has been a professor with the Department of Electrical and Electronic Engineering, Kobe University, since 2024. His current research interests include information security, cryptography, computer network, and machine learning based cyberattack analysis. He is a member of ACM and a senior member of IEICE and IPSJ.



Shoichi Saito received the BS and ME degrees in engineering from Ritsumeikan University, in 1993 and 1995, respectively, and the Dr. Eng. degree in 2000. He was a research associate with the Department of Computer and Communication Sciences, Wakayama University, in 1998. He was an assistant professor in 2003 and an associate professor in 2005. He was an associate professor with the Nagoya Institute of Technology in 2006. He has been a professor since 2016. His research interests include operating systems, security, and Internet. He is a member of ACM, IEEE CS, and IPSJ.